

PySpark / Data Engineer Interview Notes

Consolidated Q&A: duplicates merged, one clean answer with a working example for each question

Contents

PART 1: Spark Core Concepts

1. Spark Architecture and what happens when we submit a job
2. Client mode vs Cluster mode
3. Local deployment mode
4. Jobs, Stages and Tasks
5. DAG and Lineage Graph
6. How is Spark fault tolerant?
7. Narrow vs Wide transformations (and shuffle)
8. RDD vs DataFrame vs Dataset
9. SparkSession vs SparkContext
10. Catalyst Optimizer
11. Tungsten Engine
12. Predicate Pushdown and Column Pruning
13. map() vs flatMap()
14. Pandas DataFrame vs Spark DataFrame
15. Built-in functions vs UDFs (wrapper functions)
16. Global Temp View vs Temp View

PART 2: Optimization and Performance

17. Spark optimization and performance tuning techniques
18. How do you identify the bottleneck of a slow Spark job?
19. Spark configurations
20. cache() vs persist()
21. repartition() vs coalesce() vs shuffle partitions
22. Small file problem and how coalesce() helps
23. Joins and Join strategies; Broadcast Join
24. Data skew and Salting
25. Driver OOM vs Executor OOM (using Spark UI)
26. Partitioning vs Bucketing
27. Handling 10 TB of data: batch and incremental loading
28. Handling late arriving data: watermarking and incremental loading

PART 3: Storage Formats, Hive, Airflow and Window Functions

29. Parquet file format in detail
30. ORC file format in detail
31. ORC vs Parquet, and why they are preferred over other formats
32. Managed table vs External table
33. Updating records in Hive
34. Apache Airflow: architecture and workflow
35. Window functions

PART 4: PySpark and SQL Coding

36. Join Employee and Department data (CSV files)
37. Broadcast join between emp and dep
38. Rename a column (name to first_name)
39. Join types: Inner, Left, Right, Full Outer, Cross (plus Semi and Anti)
40. Join output with NULL values: how many rows?
41. Second highest salary in each department
42. Find departments with no employees
43. Second and Third highest salary (whole table), Nth highest
44. Employees earning more than their manager
45. Compare two files: employees who left, newly joined, salary changed

46. Remove duplicates (and keep the latest record)
47. Replace int values in dep, team, sub_team... with the description from table_2
48. Write a UDF (and the increment_percent example)
49. Join columns with ids 0001 and 1 without casting or substring
50. Word Count (PySpark and SQL)
51. Convert all columns (50 columns) to StringType
52. Count occurrences of values across rows (A, B, C example)
53. Take user input dynamically in a PySpark job
54. Simple spark-submit command and how to call a config file
55. SCD Type 1 and Type 2
56. CDC (Change Data Capture) implementation
57. Cumulative sum as of the last date of every month (and months with no transactions)
58. Monthly sales report including months with no sales
59. Delete records older than 30 days
60. Three consecutive absences
61. Handling double delimiters in input files
62. Identify and remove malformed records (permissive modes)
63. Process a specific file from a directory

PART 5: Linux and Shell Scripting

64. Output/effect of common commands: cat >, chmod 755, ls -ltr >
65. chmod: permission scenarios
66. Rename multiple files; shell script to rename with backup_ prefix
67. Read the first line of each of 10 files into a new file
68. grep command, searching a word in a file
69. List all files containing a specific word; print files and directories with a word
70. Commonly used Unix commands
71. How to check the status of a Spark job and its logs

PART 6: Python and Project Questions

72. Static method vs Class method in Python
73. CI/CD for a data engineering project
74. Agile methodology
75. Explaining your last project (answer framework)
76. Project folder structure

PART 1: Spark Core Concepts

1. Spark Architecture and what happens when we submit a job

Spark is a distributed engine with a **master/worker** design. One **Driver** coordinates, many **Executors** do the work, and a **Cluster Manager** hands out resources.

Component	Role
Driver	Runs your <code>main()</code> , holds the <code>SparkSession/SparkContext</code> , builds the plan (DAG), splits it into stages and tasks, schedules tasks and collects results.
Cluster Manager	Allocates CPU and memory (YARN, Kubernetes, Standalone, Mesos). It does not run your code.
Executors	JVM processes on worker nodes. They run tasks, keep cached data, and read/write shuffle files.
Task	The smallest unit of work: one task processes one partition.

What happens on submit (step by step)

- `spark-submit` contacts the cluster manager and starts the **driver** (on the client machine or inside the cluster, depending on deploy mode).
- The driver creates the **SparkSession** and asks the cluster manager for executors. Executors start and register back with the driver.
- Transformations (`select`, `filter`, `join`) are **lazy**. Spark only builds a logical plan; nothing runs yet.
- **Catalyst** optimizes the logical plan and picks a physical plan.
- An **action** (`count`, `show`, `write`) triggers a **job**. The DAG scheduler cuts it into **stages** at shuffle boundaries.
- Each stage becomes a set of **tasks** (one per partition). The task scheduler sends them to executors, preferring data locality.
- Executors run tasks, write/read shuffle data, and return results to the driver or write them to storage.
- When the application ends, the driver releases the executors.

Say in the interview: Lazy evaluation, plan optimization, DAG, then stages and tasks. Mention that the driver never processes big data itself; it only coordinates.

2. Client mode vs Cluster mode

	Client mode	Cluster mode
Driver runs on	The machine that ran <code>spark-submit</code> (edge node)	Inside the cluster (on a worker/AM container)
Best for	Interactive work, debugging, notebooks, <code>spark-shell</code>	Production batch jobs
Driver logs	Printed on your terminal	In YARN/K8s logs (<code>yarn logs -applicationId <id></code>)
If the client dies	Job is killed (driver is gone)	Job keeps running; submit session can close
Network	Slower if edge node is far from the cluster (driver-executor chatter)	Driver sits next to executors: low latency
Resource risk	Many users overload the edge node	Driver resources managed by the cluster

```
spark-submit --master yarn --deploy-mode client app.py
spark-submit --master yarn --deploy-mode cluster app.py
```

3. Local deployment mode

In **local mode** the driver and executors all run **inside one JVM on one machine**. There is no cluster manager.

- `--master local` runs 1 thread, `local[4]` runs 4 threads, `local[*]` uses all cores.
- Used for development, unit tests and small samples. No real fault tolerance or distribution.
- Other modes: **Standalone** (Spark's own manager), **YARN**, **Kubernetes**.

```
spark = SparkSession.builder.master("local[*]").appName("dev").getOrCreate()
```

4. Jobs, Stages and Tasks

Term	Meaning	Created when
Job	All work triggered by one action	Every action (<code>count</code> , <code>show</code> , <code>write</code> , <code>collect</code>)
Stage	A group of tasks that can run without a shuffle	Each wide transformation (shuffle) splits a stage
Task	Work on one partition, run by one core	Number of tasks in a stage = number of partitions

```
df = spark.read.parquet("sales")          # lazy
res = df.filter("amt > 0").groupBy("region").count() # lazy; groupBy = shuffle
res.show()                                # ACTION -> 1 job
# Stage 1: read + filter + partial aggregate (narrow) -> N tasks (N input partitions)
# Stage 2: after shuffle, final aggregate          -> spark.sql.shuffle.partitions tasks
```

5. DAG and Lineage Graph

- **DAG (Directed Acyclic Graph)**: the execution plan of a job. Nodes are operations/stages, edges are data dependencies, and there are no cycles. The DAG scheduler uses it to form stages.
- **Lineage graph**: the chain of parent RDDs/transformations that produced a dataset. It is the **recipe** Spark can replay to rebuild lost partitions.
- DAG answers "how do I run this?", lineage answers "how do I rebuild this if it is lost?".

```
rdd.toDebugString()    # prints the lineage
df.explain(True)        # logical + physical plan
```

6. How is Spark fault tolerant?

- **Lineage**: a lost partition is recomputed from its parents, so nothing is stored redundantly by default.
- **Task retries**: a failed task is rerun (`spark.task.maxFailures`, default 4) on another executor.
- **Executor/node loss**: the cluster manager starts a new executor and lost tasks and shuffle outputs are recomputed.
- **Checkpointing**: `rdd.checkpoint()` / streaming checkpoints save state to reliable storage (HDFS/S3) and cut long lineages.
- **Replicated caching**: storage levels like `MEMORY_ONLY_2` keep two copies.
- **Speculative execution**: `spark.speculation=true` relaunches very slow tasks on other nodes.
- **Driver failure**: in cluster mode YARN can restart the AM/driver (`spark.yarn.maxAppAttempts`); Structured Streaming recovers from its checkpoint.

7. Narrow vs Wide transformations (and shuffle)

	Narrow	Wide (shuffle)
Dependency	Each output partition depends on one input partition	Output partition depends on many input partitions
Data movement	None, runs in the same stage	Data moves across the network; stage boundary
Cost	Cheap, pipelined	Expensive: disk write, network, sort/hash
Examples	<code>map</code> , <code>filter</code> , <code>select</code> , <code>union</code> , <code>coalesce</code> , <code>withColumn</code>	<code>groupBy</code> , <code>reduceByKey</code> , <code>join</code> , <code>distinct</code> , <code>orderBy</code> , <code>repartition</code>

Shuffle is the redistribution of data across partitions so that rows with the same key land together. Map tasks write shuffle files to local disk, reduce tasks fetch them over the network. It is the main source of slow jobs.

How to reduce shuffle

- Filter and select columns before joins/aggregations.
- Prefer `reduceByKey/agg` over `groupByKey` (map-side combine).
- Broadcast small tables.
- Use bucketing for repeated joins on the same key.
- Enable AQE to auto-tune shuffle partitions.

8. RDD vs DataFrame vs Dataset

	RDD	DataFrame	Dataset
Abstraction	Distributed collection of objects	Rows with named columns (a table)	Typed DataFrame (JVM objects)
Schema	No	Yes	Yes
Optimizer	None (you tune it)	Catalyst + Tungsten	Catalyst + Tungsten
Type safety	Compile time	Runtime only	Compile time
Languages	All	All	Scala/Java only (not Python)
Speed	Slowest	Fast	Fast (slight serialization cost)
Use when	Unstructured data, low-level control	Almost everything (SQL-like ETL)	Scala apps needing type safety

Say in the interview: In PySpark you use DataFrames. Datasets do not exist in Python.

9. SparkSession vs SparkContext

- **SparkContext (Spark 1.x):** the original entry point and connection to the cluster. It creates RDDs and accumulators. SQL needed a separate `SQLContext`, Hive needed `HiveContext`.
- **SparkSession (Spark 2.0+):** the **unified entry point** that wraps SparkContext, SQLContext and HiveContext. Use it for DataFrames, SQL, reading data and configs.
- You can get the context from the session: `spark.sparkContext`. There is only one active SparkContext per JVM, but many sessions can share it.

```
from pyspark.sql import SparkSession
spark = SparkSession.builder.appName("demo").enableHiveSupport().getOrCreate()
sc = spark.sparkContext
```

10. Catalyst Optimizer

Catalyst is Spark SQL's **rule-based and cost-based query optimizer**. It works in four phases:

- **Analysis:** resolves column names and types against the catalog, producing a resolved logical plan.
- **Logical optimization:** rules like predicate pushdown, column pruning, constant folding, filter reordering, combining projections.
- **Physical planning:** generates candidate physical plans (for example broadcast vs sort-merge join) and picks the cheapest by cost.
- **Code generation:** whole-stage codegen produces Java bytecode (this part is shared with Tungsten).

```
df.filter("year = 2026").select("id", "amt").explain(True)
# shows: Parsed Logical Plan -> Analyzed -> Optimized Logical Plan -> Physical Plan
```

11. Tungsten Engine

Project Tungsten makes Spark's execution close to bare-metal speed. Catalyst decides **what** to run, Tungsten decides **how** to run it efficiently.

- **Off-heap memory and binary format:** data is stored in compact binary rows (**UnsafeRow**) instead of JVM objects, which means less garbage collection and less memory.
- **Cache-aware computation:** algorithms and layouts (sort, hash) are designed to use CPU L1/L2/L3 caches well.
- **Whole-stage code generation:** several operators (filter, project) are fused into one generated Java function, removing virtual calls and keeping data in registers.

Say in the interview: You can spot whole-stage codegen in `explain()` as `*(1)` in front of operators.

12. Predicate Pushdown and Column Pruning

- **Predicate pushdown:** filters are pushed down to the data source so non-matching data is skipped while reading. With Parquet/ORC, row-group min/max statistics let Spark skip whole blocks.
- **Column pruning:** only the columns that are actually used are read. This is a huge win with columnar formats.

```
df = spark.read.parquet("/data/sales").filter("country = 'IN']").select("id", "amount")
df.explain()
# FileScan parquet [id, amount]          <- ReadSchema: only 2 columns read (column pruning)
#   PushedFilters: [IsNotNull(country), EqualTo(country,IN)]  <- predicate pushdown
```

Caveat: pushdown does not work well through UDFs or on a column wrapped in a function, so write `col = 'IN'` rather than `upper(col) = 'IN'`.

13. map() vs flatMap()

- **map():** one output per input (1 to 1).
- **flatMap():** each input can produce zero, one or many outputs, and the results are flattened (1 to many).

```
rdd = sc.parallelize(["hello world", "spark is fast"])
rdd.map(lambda x: x.split(" ")).collect()      # [['hello', 'world'], ['spark', 'is', 'fast']]
rdd.flatMap(lambda x: x.split(" ")).collect()  # ['hello', 'world', 'spark', 'is', 'fast']
```

Say in the interview: DataFrame equivalent of flatMap is `explode()`. Word count is the classic use case.

14. Pandas DataFrame vs Spark DataFrame

	Pandas	Spark DataFrame
Execution	Single machine, in memory	Distributed across a cluster
Data size	Fits in one machine's RAM	GBs to PBs
Evaluation	Eager (runs immediately)	Lazy (runs on action)
Mutability	Mutable (in-place edits)	Immutable (every transformation returns a new DataFrame)
Speed on small data	Faster (no overhead)	Slower (startup, scheduling)
Fault tolerance	None	Built in
Typical use	EDA, small data, ML prep	ETL and big data pipelines

Common Pandas use cases: cleaning and exploring small datasets, quick aggregations, plotting, feature engineering before ML.

```
pdf = spark_df.limit(1000).toPandas()    # Spark -> Pandas (collects to driver: only small data!)
sdf = spark.createDataFrame(pdf)         # Pandas -> Spark
# Spark 3.2+: import pyspark.pandas as ps -> Pandas API running on Spark
```

15. Built-in functions vs UDFs (wrapper functions)

Aspect	Built-in functions	UDFs (wrapper functions)
Defined by	Spark itself	You, the user
Runs in	JVM (native)	Separate Python process
Catalyst optimization	Yes, fully optimized	No, treated as a black box
Performance	Fast	Slower (serialization JVM <-> Python)
Use case	String, date, math, aggregation	Custom logic not available as a built-in
Examples	<code>upper()</code> , <code>when()</code> , <code>datediff()</code>	Your own business-rule function

Say in the interview: Always try built-ins first, then `pandas_udf` (Arrow vectorized), and a plain Python UDF only as a last resort.

16. Global Temp View vs Temp View

	Temp view	Global temp view
Created with	<code>createOrReplaceTempView("v")</code>	<code>createGlobalTempView("v")</code>
Scope	Only the current SparkSession	Shared across all sessions in the same Spark application
Accessed as	<code>SELECT * FROM v</code>	<code>SELECT * FROM global_temp.v</code>
Lifetime	Ends with the session	Ends with the Spark application

Neither is persisted to storage. For permanent tables use `saveAsTable`.

PART 2: Optimization and Performance

17. Spark optimization and performance tuning techniques

Area	Techniques
Data layout	Use columnar formats (Parquet/ORC) with snappy; partition by low-cardinality columns (date); bucket on join keys; compact small files; keep files 128-256 MB
Reduce shuffle	Broadcast small tables; filter and select early; <code>reduceByKey/agg</code> instead of <code>groupByKey</code> ; avoid unnecessary <code>distinct/orderBy</code> ; enable AQE
Parallelism	Right-size partitions (about 128 MB each); tune <code>spark.sql.shuffle.partitions</code> ; 2-4 tasks per core
Memory and resources	5 cores per executor; set <code>executor.memory</code> and <code>memoryOverhead</code> ; dynamic allocation; avoid collecting to the driver
Code	Prefer DataFrame/SQL over RDD; built-ins over UDFs; <code>cache()</code> only DataFrames reused several times; avoid <code>withColumn</code> in long loops
Skew	AQE skew join, salting, isolate hot keys, filter null keys
Serialization	Kryo serializer for RDD-heavy jobs

Executor sizing example

Cluster: 10 nodes, each 16 cores and 64 GB. Leave 1 core and 1 GB per node for the OS/daemons, and use about 5 cores per executor.

- Executors per node = $(16 - 1) / 5 = 3$; total = 30, minus 1 for the YARN AM = **29 executors**.
- Memory per executor = $(64 - 1) / 3 =$ about 21 GB. Subtract about 10% overhead, so `--executor-memory 19g`.
- Result: `--num-executors 29 --executor-cores 5 --executor-memory 19g`.

```
spark.conf.set("spark.sql.adaptive.enabled", "true") # AQE (default on in 3.2+)
spark.conf.set("spark.sql.adaptive.coalescePartitions.enabled", "true")
spark.conf.set("spark.sql.adaptive.skewJoin.enabled", "true")
spark.conf.set("spark.sql.autoBroadcastJoinThreshold", 50 * 1024 * 1024)
```

18. How do you identify the bottleneck of a slow Spark job?

Open the **Spark UI** and go top-down: **Jobs**, then **Stages**, then **Tasks**, then the SQL tab.

What you see	Likely cause	Fix
A few tasks far slower than the median (Max >> Median)	Data skew	AQE skew join, salting, broadcast
Large Shuffle Read/Write	Too much shuffle	Broadcast, filter earlier, bucketing
Spill (Memory/Disk) is high	Partitions too big for memory	More partitions, more executor memory
GC Time is red/high	Memory pressure, too many objects	More memory, fewer cores per executor, avoid UDFs
Thousands of very short tasks	Too many small partitions/files	Coalesce, compaction, fewer shuffle partitions
Few long tasks, idle cores	Too few partitions	Repartition to 2-4x total cores
Long scheduler delay	Too many tasks	Reduce partitions

What you see	Likely cause	Fix
Executors idle waiting	Reading from slow source	Increase read parallelism, push filters down

- **SQL tab:** inspect the plan. Look for a sort-merge join that should be broadcast, and for full scans with no pushed filters.
- **Executors tab:** GC time, memory used, dead executors. **Storage tab:** cached data size.

19. Spark configurations

Config	Purpose
<code>spark.executor.memory / spark.executor.cores</code>	Heap size and cores per executor
<code>spark.executor.memoryOverhead</code>	Off-heap memory (Python workers, buffers); a common cause of YARN kills
<code>spark.driver.memory</code>	Driver heap
<code>spark.sql.shuffle.partitions</code>	Partitions after a shuffle (default 200)
<code>spark.default.parallelism</code>	Default partitions for RDD operations
<code>spark.sql.autoBroadcastJoinThreshold</code>	Max table size to auto-broadcast (default 10 MB)
<code>spark.sql.adaptive.enabled</code>	Adaptive Query Execution
<code>spark.dynamicAllocation.enabled</code>	Scale executors with the workload
<code>spark.serializer</code>	Use <code>KryoSerializer</code> for speed

Where to set them (precedence high to low): code `SparkConf/spark.conf.set` > `spark-submit --conf` > `spark-defaults.conf`.

20. cache() vs persist()

- `cache()` is shorthand for `persist()` with the **default storage level** (`MEMORY_AND_DISK` for DataFrames, `MEMORY_ONLY` for RDDs).
- `persist(level)` lets you **choose** the storage level.
- Both are lazy. Data is stored the first time an action runs. Remove it with `unpersist()`.

Storage level	Meaning
<code>MEMORY_ONLY</code>	Keep in memory; partitions that do not fit are recomputed
<code>MEMORY_AND_DISK</code>	Spill to disk what does not fit
<code>DISK_ONLY</code>	Disk only
<code>MEMORY_ONLY_SER / MEMORY_AND_DISK_SER</code>	Serialized: smaller but slower to read
<code>..._2</code> (for example <code>MEMORY_ONLY_2</code>)	Replicate on 2 nodes for faster recovery

```

from pyspark import StorageLevel
df.cache()                # same as persist(MEMORY_AND_DISK)
df.persist(StorageLevel.DISK_ONLY)
df.count()                # action materializes the cache
df.unpersist()

```

Say in the interview: Cache only when a DataFrame is reused by multiple actions. Caching something used once wastes memory.

21. repartition() vs coalesce() vs shuffle partitions

	repartition(n)	coalesce(n)
Shuffle	Yes (full shuffle)	No (merges existing partitions)
Increase partitions	Yes	No
Decrease partitions	Yes	Yes (preferred)
Partition sizes	Balanced	Can be uneven
Cost	Higher	Cheaper

How do you repartition data?

```
df.repartition(100)           # to 100 partitions, round-robin
df.repartition("country")     # hash partition by column (spark.sql.shuffle.partitions)
df.repartition(50, "country", "year") # N partitions by columns
df.repartitionByRange(20, "order_date") # range partitioning (good for sorted output)
df.coalesce(10)               # reduce without shuffle
df.rdd.getNumPartitions()      # check current count
```

Repartitioning vs shuffle partitions

- **Repartitioning** is something you do explicitly with `repartition()/coalesce()` to control layout.
- **Shuffle partitions** (`spark.sql.shuffle.partitions`, default 200) is the number of partitions Spark automatically creates after a wide transformation (join, groupBy). You do not call anything; it is a config. AQE can merge small ones automatically.

Rule of thumb: target roughly 128 MB per partition, and 2-4 partitions per core.

22. Small file problem and how coalesce() helps

Problem: thousands of tiny files (KBs to a few MBs) instead of fewer files near the block size (128-256 MB).

- **Metadata overhead:** the HDFS NameNode (or S3 listing) tracks every file.
- **Too many tasks:** roughly one task per file, so scheduling cost exceeds real work.
- **Slow reads:** opening and closing many files is expensive.

Common causes: default 200 shuffle partitions writing 200 tiny files, `partitionBy` on a high-cardinality column, streaming micro-batches.

How coalesce() helps: it merges existing partitions **without a full shuffle**, so fewer and larger files are written.

```
df.coalesce(10).write.mode("overwrite").parquet("/out/path")
df.write.option("maxRecordsPerFile", 1000000).parquet("/out/path") # cap file size by rows
```

Other fixes: compaction jobs, Delta `OPTIMIZE`, AQE partition coalescing. Caveat: `coalesce(1)` before heavy work can bottleneck one task; use `repartition(n)` if you need balanced sizes.

23. Joins and Join strategies; Broadcast Join

Spark picks one of these physical strategies:

Strategy	How it works	When used
Broadcast Hash Join	Small table is sent to every executor; big table is not shuffled	One side is below the broadcast threshold
Sort Merge Join	Both sides are shuffled and sorted by key, then merged	Default for two large tables
Shuffle Hash Join	Shuffle both, build a hash table on the smaller side	One side much smaller but too big to broadcast
Broadcast Nested Loop / Cartesian	Compare every row pair	Cross joins and non-equi joins

Broadcast join: explanation and use cases

The small table is **collected to the driver and shipped once to every executor**, so the large table never shuffles. This is usually the biggest single speedup for join-heavy jobs.

- **Use when:** fact table joined to a small dimension/lookup table (country codes, departments, product master).
- **Limit:** the table must fit in driver and executor memory. Default threshold is 10 MB (`spark.sql.autoBroadcastJoinThreshold`).
- **Avoid when:** both tables are large (OOM risk) or the small table changes every run unpredictably.

```
from pyspark.sql.functions import broadcast
result = big_df.join(broadcast(small_df), "dept_id", "left")
# SQL hint: SELECT /*+ BROADCAST(d) */ * FROM emp e JOIN dep d ON e.dep_id = d.dep_id
```

24. Data skew and Salting

Skew means some keys have far more rows than others, so a few tasks do most of the work and the stage waits for them. Symptom in Spark UI: Max task time and shuffle read are much larger than the median.

Ways to handle it

- **AQE skew join:** `spark.sql.adaptive.skewJoin.enabled=true` splits big partitions automatically (Spark 3+).
- **Broadcast** the smaller side if possible.
- **Filter or isolate** hot keys (for example NULL keys) and process them separately.
- **Salting:** add a random suffix to the hot key so its rows spread over many partitions.

Salting example (skewed big table joined to a smaller one)

```
from pyspark.sql import functions as F
N = 10 # number of salt buckets

big = big.withColumn("salt", (F.rand() * N).cast("int")) # random salt on big side
small = small.withColumn("salt", F.explode(F.array([F.lit(i) for i in range(N)]))) # replicate

joined = (big.join(small, ["key", "salt"], "inner") # key now spread over N partitions
          .drop("salt"))
```

Say in the interview: Salting trades extra data (the small side is replicated N times) for balanced work. For aggregations, do a two-stage aggregate: partial by (key, salt), then final by key.

25. Driver OOM vs Executor OOM (using Spark UI)

	Driver OOM	Executor OOM
Where to look	Executors tab, "driver" row; driver logs	Stages tab (failed tasks), Executors tab
Symptoms	<code>OutOfMemoryError: Java heap space</code> in driver log; job dies at an action	<code>ExecutorLostFailure</code> , "Container killed by YARN", exit code 137/143; tasks retry
Causes	<code>collect()</code> , <code>toPandas()</code> , huge broadcast, too many partitions, big <code>maxResultSize</code>	Skew, large partitions, big shuffles, heavy UDFs, too many cores per executor
UI evidence	Driver memory near max, high GC	Max >> Median task time, high Spill, high GC time
Fix	Avoid collect, raise <code>spark.driver.memory</code> , lower broadcast threshold	Fix skew, more partitions, raise <code>executor.memory/memoryOverhead</code> , fewer cores per executor

Say in the interview: If a few tasks are far slower/bigger than the rest, it is skew on the executor side. If the app dies when pulling data to the driver, it is driver OOM.

26. Partitioning vs Bucketing

	Partitioning	Bucketing
What	Splits data into directories by column value	Splits data into a fixed number of files by hash of a column
Column type	Low cardinality (date, country)	High cardinality (customer_id, user_id)
Benefit	Partition pruning: queries skip whole folders	Shuffle-free joins and aggregations on the bucket key; even file sizes
Risk	Too many partitions = small file problem	Must be written with the same bucket count/column to benefit
Layout	<code>/sales/country=IN/year=2026/</code>	<code>part-0000..part-0007</code> inside each partition

```
(df.write.partitionBy("country", "year")
  .bucketBy(8, "customer_id").sortBy("customer_id")
  .format("parquet").mode("overwrite").saveAsTable("sales_bucketed"))
```

```
spark.sql("SELECT * FROM sales_bucketed WHERE country = 'IN'") # reads only /country=IN/
```

`bucketBy` only works with `saveAsTable`, not a plain path write.

27. Handling 10 TB of data: batch and incremental loading

Batch processing

- Store input as **partitioned Parquet/ORC**, not CSV/JSON.
- Process in **chunks** (by date partition) so a failure only reprocesses one chunk.
- Size the cluster by partitions: 10 TB / 128 MB is roughly 80,000 partitions. Give enough cores and use dynamic allocation.
- Enable AQE, broadcast dimension tables, filter and select columns first (pushdown and pruning), and avoid `collect()`.
- Control output file size and write partitioned by date. Make jobs **idempotent** (overwrite the partition, not append blindly).

Incremental loading

- Do not reload 10 TB every run. Load only **new/changed** rows using a watermark column (**updated_at**), an ingestion date partition, or CDC logs.
- Keep the last processed value in a **control/audit table**.
- Upsert into the target with **MERGE** (Delta/Iceberg/Hudi).

```
last_ts = spark.table("ctl.watermark").filter("tbl = 'orders'").first()["last_ts"]
delta = spark.table("src.orders").filter(F.col("updated_at") > last_ts)

delta.createOrReplaceTempView("delta")
spark.sql("""
    MERGE INTO dw.orders t USING delta s ON t.order_id = s.order_id
    WHEN MATCHED THEN UPDATE SET *
    WHEN NOT MATCHED THEN INSERT *
""")
# then update ctl.watermark with max(updated_at) from delta
```

28. Handling late arriving data: watermarking and incremental loading

Late data is an event that arrives after the time it belongs to (a mobile event from 10:00 arriving at 10:20).

Structured Streaming: watermark

A **watermark** says "wait this long for late events, then drop anything older". Spark keeps state only for the allowed lateness, so state does not grow forever.

```
from pyspark.sql import functions as F
agg = (events
    .withWatermark("event_time", "10 minutes")          # tolerate 10 min lateness
    .groupBy(F.window("event_time", "5 minutes"), "user_id")
    .count())
agg.writeStream.outputMode("update").option("checkpointLocation", "/chk/a").start()
```

Batch: incremental loading with a lookback window

- Each run reprocesses a **lookback window** (for example the last 3 days) instead of only today.
- Upsert with **MERGE** so reprocessed rows replace old versions rather than duplicating.
- Partition by **event date** (not ingestion date) so late rows land in the correct partition; overwrite only the affected partitions.

PART 3: Storage Formats, Hive, Airflow and Window Functions

29. Parquet file format in detail

Parquet is an open-source, **columnar**, binary file format from the Hadoop ecosystem and the default for Spark.

- **Layout:** file > **row groups** (about 128 MB) > **column chunks** > **pages**. A **footer** stores the schema and per-column min/max statistics.
- **Columnar storage:** values of a column are stored together, so queries read only the needed columns (column pruning).
- **Predicate pushdown:** footer statistics let Spark skip row groups that cannot match a filter.
- **Encoding and compression:** dictionary, run-length and bit-packing encodings plus snappy/gzip/zstd. Typically 5-10x smaller than CSV.
- **Schema and nesting:** self-describing, supports nested structures (structs, arrays, maps) and schema evolution (adding columns).
- **Best for:** analytics that read a few columns of many rows; works across Spark, Hive, Presto, Impala.

```
df.write.option("compression", "snappy").partitionBy("year").parquet("/data/sales")
spark.read.option("mergeSchema", "true").parquet("/data/sales") # merge evolved schemas
```

30. ORC file format in detail

ORC (Optimized Row Columnar) is a columnar format created for **Hive**. It is also splittable, compressed and self-describing.

- **Layout:** file > **stripes** (about 64-256 MB) > index data, row data, stripe footer. A **file footer** holds statistics and the schema.
- **Built-in indexes:** min/max and optional **bloom filters** every 10,000 rows allow aggressive row skipping.
- **Compression:** zlib by default (higher ratio) or snappy (faster).
- **ACID support:** ORC is the format required for Hive transactional (update/delete) tables.
- **Best for:** Hive-centric warehouses and workloads that need ACID.

```
df.write.option("compression", "zlib").orc("/data/sales_orc")
spark.read.orc("/data/sales_orc")
```

31. ORC vs Parquet, and why they are preferred over other formats

	ORC	Parquet
Origin / best fit	Hive / Hortonworks; Hive ecosystem	Twitter + Cloudera; Spark and everything else
Internal unit	Stripes	Row groups
Compression	Usually slightly better (zlib)	Very good (snappy default)
ACID / updates	Yes (Hive transactional tables)	No (use Delta/Iceberg on top)
Indexes	Built-in index + bloom filter	Min/max stats (bloom filters in newer versions)
Nested data	Supported, less efficient	Excellent
Spark support	Good	Best, the default
Schema evolution	Good	Good

Why columnar formats beat CSV/JSON/Avro for analytics

- **Less I/O:** only the needed columns are read; filters skip blocks (pruning and pushdown).
- **Smaller:** similar values stored together compress and encode very well.
- **Faster:** schema and types are stored, so no parsing and no inference.
- **Splittable and parallel:** large files can be read in parallel by many tasks.
- Row formats like **Avro** are better for write-heavy/streaming and record-at-a-time use; CSV/JSON are human-readable but slow and large.

32. Managed table vs External table

	Managed (internal) table	External table
Data location	Warehouse directory controlled by Hive/Spark	Any path you specify with LOCATION
DROP TABLE	Deletes metadata and data	Deletes only metadata ; files stay
Ownership	The metastore owns the data	You/other tools own the data
Use when	Staging/temporary tables, Hive fully owns the data	Shared data, raw/landing zone, data used by other tools

```
CREATE TABLE sales_m (id INT, amt DOUBLE) STORED AS PARQUET;           -- managed
CREATE EXTERNAL TABLE sales_e (id INT, amt DOUBLE) STORED AS PARQUET
LOCATION '/data/sales';                                                -- external
```

33. Updating records in Hive

Classic Hive tables are write-once. Row-level **UPDATE/DELETE/MERGE** needs **ACID transactional tables**.

Required file format and table properties

- File format must be **ORC**.
- Table must have **TBLPROPERTIES ('transactional'='true')**.
- Hive 3+ makes managed ORC tables ACID by default.

Required Hive configuration

```
SET hive.support.concurrency = true;
SET hive.txn.manager = org.apache.hadoop.hive.ql.lockmgr.DbTxnManager;
SET hive.exec.dynamic.partition.mode = nonstrict;
-- on the metastore side:
-- hive.compactor.initiator.on = true
-- hive.compactor.worker.threads = 1 (or more)

CREATE TABLE emp_acid (id INT, name STRING, sal INT)
STORED AS ORC TBLPROPERTIES ('transactional'='true');

UPDATE emp_acid SET sal = 5000 WHERE id = 1;
DELETE FROM emp_acid WHERE id = 2;
```

Updates write **delta files**; a background **compactor** merges them. **Alternatives without ACID:** **INSERT OVERWRITE** the partition with the corrected data, or use Delta/Iceberg/Hudi with **MERGE**.

34. Apache Airflow: architecture and workflow

Airflow is a workflow orchestrator where pipelines are defined as Python code. A workflow is a **DAG** (Directed Acyclic Graph) of tasks.

Component	Role
Web server	UI to view DAGs, runs, logs, trigger and clear tasks
Scheduler	Parses DAG files, decides when tasks are due, queues them
Executor	How tasks run: Local, Celery, Kubernetes
Workers	Processes/pods that actually run the tasks
Metadata database	Postgres/MySQL storing DAG state, task instances, connections, variables
DAGs folder	Python files the scheduler reads

Key concepts

- **Operator:** a template for a task (BashOperator, PythonOperator, SparkSubmitOperator). **Task** is an operator instance.
- **Dependencies:** `a >> b >> c`. **Retries, SLAs, schedule, catchup** are set per DAG/task.
- **Sensor:** waits for a condition (file arrival). **XCom:** small data passed between tasks. **Connections/Variables:** credentials and config.

Workflow of a run

Scheduler creates a DAG run at the schedule time, queues ready tasks, the executor hands them to workers, workers update state in the metadata DB, and the UI shows it. Failed tasks retry; downstream tasks wait on upstream success.

```
from datetime import datetime, timedelta
from airflow import DAG
from airflow.operators.bash import BashOperator

with DAG("daily_sales_etl", start_date=datetime(2026, 1, 1), schedule="@daily",
        catchup=False, default_args={"retries": 2, "retry_delay": timedelta(minutes=5)}) as dag:

    extract = BashOperator(task_id="extract", bash_command="python extract.py {{ ds }}")
    transform = BashOperator(
        task_id="transform",
        bash_command="spark-submit --master yarn --deploy-mode cluster etl.py {{ ds }}"
    )
    load = BashOperator(task_id="load", bash_command="python load.py {{ ds }}")

    extract >> transform >> load
```

35. Window functions

A window function computes a value **for each row using a set of related rows** (the window) without collapsing rows like `GROUP BY` does.

Syntax: `function()` `OVER (PARTITION BY ... ORDER BY ... ROWS BETWEEN ...)`

Type	Functions	Purpose
Ranking	<code>row_number</code> , <code>rank</code> , <code>dense_rank</code> , <code>ntile</code>	Number/rank rows within a partition
Analytic (offset)	<code>lag</code> , <code>lead</code> , <code>first_value</code> , <code>last_value</code>	Look at previous/next rows
Aggregate	<code>sum</code> , <code>avg</code> , <code>min</code> , <code>max</code> , <code>count</code> over a window	Running totals, moving averages

rank vs dense_rank vs row_number (salaries 100, 100, 90)

salary	row_number	rank	dense_rank
100	1	1	1
100	2	1	1
90	3	3	2

```
from pyspark.sql import functions as F
from pyspark.sql.window import Window
```

```
w = Window.partitionBy("dep_id").orderBy(F.desc("sal"))
df.select("*",
        F.row_number().over(w).alias("rn"),
        F.dense_rank().over(w).alias("dr"),
        F.lag("sal", 1).over(w).alias("prev_sal"),
        F.sum("sal").over(Window.partitionBy("dep_id")).alias("dep_total"),
        F.sum("sal").over(Window.partitionBy("dep_id").orderBy("emp_id")
                          .rowsBetween(Window.unboundedPreceding, Window.currentRow)).alias("running"))
```

```
SELECT emp_id, dep_id, sal,
       ROW_NUMBER() OVER (PARTITION BY dep_id ORDER BY sal DESC) AS rn,
       LAG(sal)      OVER (PARTITION BY dep_id ORDER BY sal DESC) AS prev_sal,
       SUM(sal)      OVER (PARTITION BY dep_id ORDER BY emp_id
                          ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS running
FROM emp;
```

PART 4: PySpark and SQL Coding

All coding answers below use the same small sample data, so you can run and explain them one by one. **emp** and **dep** are used for join, ranking and comparison questions.

```
from pyspark.sql import SparkSession, functions as F
from pyspark.sql.window import Window
spark = SparkSession.builder.appName("interview").getOrCreate()

emp = spark.createDataFrame(
    [(1, "aa", 2, 1000), (2, "bb", 2, 3000), (3, "cc", 2, 2000),
     (22, "ss", 5, 2000), (23, "tt", 5, 4000), (24, "uu", 5, 3000), (25, "vv", 1, 1500)],
    ["emp_id", "name", "dep_id", "sal"])
dep = spark.createDataFrame([(1, "HR"), (2, "IT"), (3, "Finance")], ["dep_id", "dep_name"])
emp.createOrReplaceTempView("emp"); dep.createOrReplaceTempView("dep")
# dep 3 has no employees; dep 5 (used by employees) is missing from dep
```

36. Join Employee and Department data (CSV files)

```
emp = spark.read.option("header", True).option("inferSchema", True).csv("/data/emp.csv")
dep = spark.read.option("header", True).option("inferSchema", True).csv("/data/dep.csv")

# joining on the column name avoids a duplicate dep_id column
joined = emp.join(dep, on="dep_id", how="inner")
joined.show()
```

In production, pass an explicit schema instead of **inferSchema** (inference reads the file twice and may guess wrong types).

37. Broadcast join between emp and dep

```
from pyspark.sql.functions import broadcast
emp.join(broadcast(dep), "dep_id", "left").show()
emp.join(broadcast(dep), "dep_id").explain() # look for BroadcastHashJoin in the plan

SELECT /*+ BROADCAST(d) */ e.*, d.dep_name
FROM emp e LEFT JOIN dep d ON e.dep_id = d.dep_id;
```

38. Rename a column (name to first_name)

```
emp.withColumnRenamed("name", "first_name") # one column
emp.withColumnsRenamed({"name": "first_name", "sal": "salary"}) # many (Spark 3.4+)
emp.select("emp_id", F.col("name").alias("first_name")) # while selecting
emp.toDF("emp_id", "first_name", "dep_id", "sal") # rename ALL by position

SELECT emp_id, name AS first_name FROM emp;
ALTER TABLE emp RENAME COLUMN name TO first_name; -- table-level (Delta/Iceberg)
```

39. Join types: Inner, Left, Right, Full Outer, Cross (plus Semi and Anti)

Join	Returns	Rows with the sample data
Inner	Only matching rows	4 (emp 1, 2, 3, 25)
Left	All emp rows, dep columns NULL if no match	7 (dep 5 gives NULL dep_name)
Right	All dep rows, emp columns NULL if no match	5 (dep 3 Finance has NULL emp)
Full outer	All rows from both sides	8
Cross	Every emp row with every dep row	7 x 3 = 21
Left semi	emp rows that have a match (only emp columns)	4
Left anti	emp rows with no match	3 (emp 22, 23, 24)

```
emp.join(dep, "dep_id", "inner")
emp.join(dep, "dep_id", "left")      # same as "left_outer"
emp.join(dep, "dep_id", "right")
emp.join(dep, "dep_id", "full")     # "outer" / "full_outer"
emp.crossJoin(dep)
emp.join(dep, "dep_id", "left_semi")
emp.join(dep, "dep_id", "left_anti")

SELECT * FROM emp e INNER JOIN dep d ON e.dep_id = d.dep_id;
SELECT * FROM emp e LEFT JOIN dep d ON e.dep_id = d.dep_id;
SELECT * FROM emp e RIGHT JOIN dep d ON e.dep_id = d.dep_id;
SELECT * FROM emp e FULL OUTER JOIN dep d ON e.dep_id = d.dep_id;
SELECT * FROM emp CROSS JOIN dep;
```

40. Join output with NULL values: how many rows?

data1 = {1, 1, 1, NULL, NULL} and data2 = {1, NULL, 1}. **NULL never equals NULL** in a join condition, so NULLs do not match each other.

Join	Calculation	Rows
Inner	three 1s x two 1s = 6 matches (NULLs do not match)	6
Left	6 matches + 2 unmatched NULL rows from data1	8
Right	6 matches + 1 unmatched NULL row from data2	7
Full outer	6 + 2 + 1	9
Cross	5 x 3	15
Inner, null-safe (<=>)	6 + (2 NULLs x 1 NULL) = 8	8

```
d1 = spark.createDataFrame([(1,),(1,),(1,),(None,),(None,)], "v int")
d2 = spark.createDataFrame([(1,),(None,),(1,)], "v int")
d1.join(d2, "v", "inner").count()      # 6
d1.join(d2, "v", "left").count()      # 8
d1.alias("a").join(d2.alias("b"), F.col("a.v").eqNullSafe(F.col("b.v"))).count() # 8
```

41. Second highest salary in each department

```
w = Window.partitionBy("dep_id").orderBy(F.desc("sal"))
(emp.withColumn("rnk", F.dense_rank().over(w))
 .filter("rnk = 2")
 .select("dep_id", "emp_id", "name", "sal").show())
# dep 2 -> 2000 (cc), dep 5 -> 3000 (uu). dep 1 has only one employee, so it has no second highest.
```

```
SELECT dep_id, emp_id, name, sal FROM (
  SELECT *, DENSE_RANK() OVER (PARTITION BY dep_id ORDER BY sal DESC) AS rnk FROM emp
) t WHERE rnk = 2;
```

Say in the interview: Use DENSE_RANK so two people with the top salary do not hide the second distinct value. Use ROW_NUMBER only if you want exactly one row.

42. Find departments with no employees

```
dep.join(emp, "dep_id", "left_anti") # best: anti join
dep.join(emp, "dep_id", "left").filter(F.col("emp_id").isNull()) # left join + IS NULL

SELECT d.* FROM dep d WHERE NOT EXISTS (SELECT 1 FROM emp e WHERE e.dep_id = d.dep_id);
SELECT d.* FROM dep d LEFT JOIN emp e ON d.dep_id = e.dep_id WHERE e.dep_id IS NULL;
```

Result: dep 3 (Finance). **Trap:** `NOT IN (SELECT dep_id FROM emp)` returns **no rows** if the subquery contains a NULL, so prefer `NOT EXISTS` or an anti join.

43. Second and Third highest salary (whole table), Nth highest

```
SELECT MAX(sal) AS second_highest FROM emp WHERE sal < (SELECT MAX(sal) FROM emp);

SELECT DISTINCT sal FROM (
  SELECT sal, DENSE_RANK() OVER (ORDER BY sal DESC) AS rnk FROM emp
) t WHERE rnk = 3; -- third highest; change 3 to any N

N = 3
nth = df.select("sal").distinct().orderBy(F.desc("sal")).limit(N).collect()[-1]["sal"]
```

A window with no `PARTITION BY` moves all data to one partition, so for huge tables prefer the `distinct + limit` approach. `MAX(...) < MAX(...)` returns NULL when there is no second value.

44. Employees earning more than their manager

```
-- employee(id, name, salary, manager_id)
SELECT e.name AS employee, e.salary, m.name AS manager, m.salary AS manager_salary
FROM employee e
JOIN employee m ON e.manager_id = m.id
WHERE e.salary > m.salary;

e, m = employee.alias("e"), employee.alias("m")
e.join(m, F.col("e.manager_id") == F.col("m.id")).filter("e.salary > m.salary") \
  .select("e.name", "e.salary", F.col("m.name").alias("manager"),
         F.col("m.salary").alias("mgr_salary"))
```

This is a **self join**: the same table plays two roles (employee and manager).

45. Compare two files: employees who left, newly joined, salary changed

```
old = spark.createDataFrame([(1, "aa", 1000), (2, "bb", 2000), (3, "cc", 3000)], ["emp_id", "name", "sal"])
new = spark.createDataFrame([(1, "aa", 1000), (2, "bb", 2500), (4, "dd", 4000)], ["emp_id", "name", "sal"])

left_org = old.join(new, "emp_id", "left_anti") # in old, not in new -> emp 3
joined = new.join(old, "emp_id", "left_anti") # in new, not in old -> emp 4
changed = (old.alias("o").join(new.alias("n"), "emp_id")
          .filter("o.sal <> n.sal")
          .select("emp_id", F.col("o.sal").alias("old_sal"),
                 F.col("n.sal").alias("new_sal"))) # emp 2
```

```

SELECT 'LEFT' AS status, o.emp_id FROM emp_old o LEFT ANTI JOIN emp_new n ON o.emp_id = n.emp_id
UNION ALL
SELECT 'NEW', n.emp_id FROM emp_new n LEFT ANTI JOIN emp_old o ON n.emp_id = o.emp_id
UNION ALL
SELECT 'SALARY_CHANGED', n.emp_id
FROM emp_old o JOIN emp_new n ON o.emp_id = n.emp_id WHERE o.sal <> n.sal;

```

A single **full outer join** with a **CASE** expression also works and is the base of the CDC pattern later in this part.

46. Remove duplicates (and keep the latest record)

Exact duplicate rows

```

df.distinct()                # whole-row duplicates
df.dropDuplicates(["id"])    # by key; keeps an arbitrary row per id

```

Keep the latest record per key (window function)

```

w = Window.partitionBy("id").orderBy(F.col("updated_at").desc())
df.withColumn("rn", F.row_number().over(w)).filter("rn = 1").drop("rn")

SELECT * FROM (
  SELECT *, ROW_NUMBER() OVER (PARTITION BY id ORDER BY updated_at DESC) rn FROM t
) x WHERE rn = 1;

```

Without a window function or dropDuplicates()

```

# A) max(struct): struct compares by its first field, so put updated_at first
(df.groupBy("id")
  .agg(F.max(F.struct("updated_at", "name", "sal")).alias("r"))
  .select("id", "r.*"))

# B) self join on max timestamp
latest = df.groupBy("id").agg(F.max("updated_at").alias("updated_at"))
df.join(latest, ["id", "updated_at"])

SELECT DISTINCT * FROM t;
SELECT * FROM t UNION SELECT * FROM t;          -- UNION removes duplicates

SELECT t.* FROM t
JOIN (SELECT id, MAX(updated_at) AS mx FROM t GROUP BY id) m
  ON t.id = m.id AND t.updated_at = m.mx;

```

Option B can return two rows if two records share the same latest timestamp, so add a tie-breaker if needed.

47. Replace int values in dep, team, sub_team... with the description from table_2

Data

table_1: name	id	dep	team	sub_team
Tharun	1	23	5	10
kiran	2	21	6	11

table_2: value	id (category)	desc
23	department	dep_name_1
21	department	dep_name_2
5	team	team_1
10	sub_team	sub_team_1

`table_2.id` holds the **category** (which column the value belongs to) and `value` holds the code. The same number can exist in two categories, so always match on the category too. Because `table_1` has about 100 columns, avoid a chain of 100 joins; use one **lookup map per column** and a single `select`.

```
from itertools import chain

# 1) small lookup -> python dict: {category: {value: desc}}
lookup = {}
for r in table_2.collect():
    lookup.setdefault(r["id"], {})[r["value"]] = r["desc"]

# 2) which table_1 column uses which category (extend to all 100 columns)
col_to_cat = {"dep": "department", "team": "team", "sub_team": "sub_team"}

def to_map(d):
    return F.create_map(*[F.lit(x) for x in chain(*d.items())])

exprs = []
for c in table_1.columns:
    # keeps original column order
    if c in col_to_cat:
        m = to_map(lookup[col_to_cat[c]])
        # keep the original code if there is no lookup match
        exprs.append(F.coalesce(m[F.col(c)], F.col(c).cast("string")).alias(c))
    else:
        exprs.append(F.col(c))

result = table_1.select(*exprs)
result.show()
# Tharun | 1 | dep_name_1 | team_1 | sub_team_1
# kiran  | 2 | dep_name_2 | 6      | 11      (6 and 11 have no lookup row)
```

Alternative: broadcast join per column

```
df = table_1
for c, cat in col_to_cat.items():
    lk = F.broadcast(table_2.filter(F.col("id") == cat)
                      .select(F.col("value").alias(c), F.col("desc").alias(c + "_d")))
    df = df.join(lk, c, "left").drop(c).withColumnRenamed(c + "_d", c)
df = df.select(table_1.columns)
```

Say in the interview: Map approach: no shuffle, one projection, scales to 100 columns. Join approach: works if the lookup is too large to collect, but each join deepens the plan.

48. Write a UDF (and the increment_percent example)

A **UDF (User Defined Function)** wraps your own Python logic so you can use it on DataFrame columns or in SQL. Steps: define a function, register it with a return type, apply it.

```
from pyspark.sql.types import DoubleType

increment_percent = 10

def new_salary(sal, pct):
    if sal is None: return None
    return float(sal) * (1 + pct / 100.0)

inc_udf = F.udf(new_salary, DoubleType())
# pass the parameter as a literal column
emp.withColumn("new_sal", inc_udf("sal", F.lit(increment_percent))).show()

spark.udf.register("new_sal_sql", new_salary, DoubleType())
spark.sql("SELECT emp_id, new_sal_sql(sal, 10) FROM emp")
```

Better options

```
# 1) Built-in expression (best: Catalyst can optimize it)
emp.withColumn("new_sal", F.col("sal") * (1 + F.lit(increment_percent) / 100))

# 2) pandas_udf (vectorized with Arrow, much faster than a row-at-a-time UDF)
import pandas as pd
from pyspark.sql.functions import pandas_udf
@pandas_udf("double")
def inc_pd(s: pd.Series) -> pd.Series:
    return s * (1 + increment_percent / 100)
```

Say in the interview: UDFs are black boxes to Catalyst (no optimization) and cost Python-JVM serialization. Prefer built-ins, then `pandas_udf`, then a plain UDF.

49. Join columns with ids 0001 and 1 without casting or substring

Problem: `0001`, `001` and `1` should match but are different strings and the number of leading zeros is inconsistent. Normalize both keys by **stripping leading zeros** with a regex or `TRIM(LEADING ...)`, then join on the normalized key.

```
a = a.withColumn("jk", F.regexp_replace("id", r"^0+(?!$)", ""))      # (?!$) keeps a lone "0"
b = b.withColumn("jk", F.regexp_replace("id", r"^0+(?!$)", ""))
a.join(b, "jk", "inner")

SELECT * FROM a JOIN b
  ON regexp_replace(a.id, '^0+(?!$)', '') = regexp_replace(b.id, '^0+(?!$)', '');

-- alternative: TRIM(LEADING '0' FROM a.id) = TRIM(LEADING '0' FROM b.id)
-- (turns '000' into an empty string, so use the regex form if all-zero ids exist)
```

Computing the key on both sides forces a shuffle but is correct. If you can, clean the id once at ingestion and store the normalized key.

50. Word Count (PySpark and SQL)

```
# RDD
counts = (spark.sparkContext.textFile("input.txt")
          .flatMap(lambda l: l.lower().split())
          .map(lambda w: (w, 1))
          .reduceByKey(lambda a, b: a + b))

# DataFrame
(spark.read.text("input.txt")
 .select(F.explode(F.split(F.lower("value"), r"\s+")).alias("word"))
 .filter("word <> ''")
 .groupBy("word").count().orderBy(F.desc("count")).show())

SELECT word, COUNT(*) AS cnt
FROM (SELECT explode(split(lower(value), '\s+')) AS word FROM lines) t
WHERE word <> ''
GROUP BY word ORDER BY cnt DESC;
```

51. Convert all columns (50 columns) to StringType

```
df_str = df.select([F.col(c).cast("string").alias(c) for c in df.columns])
```

A single `select` is better than a `withColumn` loop: 50 `withColumn` calls build a deep logical plan and slow down analysis.

52. Count occurrences of values across rows (A, B, C example)

Input lines `A, B, C / A, C / B, A` should give A 3, B 2, C 2. Split each line, explode into one row per item, then group and count.

```
df = spark.createDataFrame([("A, B, C",), ("A, C",), ("B, A",)], ["line"])
(df.select(F.explode(F.split("line", r",\s*")).alias("item"))
 .groupBy("item").count().orderBy("item").show())
# A 3 | B 2 | C 2

SELECT item, COUNT(*) AS cnt
FROM (SELECT explode(split(line, ',\s*')) AS item FROM lines) t
GROUP BY item ORDER BY item;

sc.parallelize(["A, B, C", "A, C", "B, A"]).flatMap(lambda l: [x.strip() for x in l.split(",")]) \
 .map(lambda x: (x, 1)).reduceByKey(lambda a, b: a + b).collect()
```

53. Take user input dynamically in a PySpark job

Batch jobs run unattended, so avoid `input()`. Pass values as **arguments or configuration**.

```
import argparse
parser = argparse.ArgumentParser()
parser.add_argument("--run_date", required=True)
parser.add_argument("--env", default="dev")
args = parser.parse_args()

df = spark.read.parquet(f"/data/{args.env}/sales").filter(F.col("dt") == args.run_date)

spark-submit job.py --run_date 2026-10-07 --env prod
# read inside the job with spark.conf.get("spark.myapp.run_date")
spark-submit --conf spark.myapp.run_date=2026-10-07 job.py
```

- **sys.argv** works for simple positional arguments (`sys.argv[1]`).
- **Environment variables:** `os.environ.get("RUN_DATE")`.
- **Orchestrator:** Airflow passes `{{ ds }}` into the command; Databricks uses widgets/job parameters.
- `input()` only works in client mode or an interactive shell/notebook, never in cluster mode.

54. Simple spark-submit command and how to call a config file

```
spark-submit \
  --master yarn \
  --deploy-mode cluster \
  --name my_etl_job \
  --driver-memory 4g \
  --executor-memory 8g \
  --executor-cores 4 \
  --num-executors 10 \
  --conf spark.sql.shuffle.partitions=200 \
  --py-files deps.zip \
  --files /path/config.json \
  my_job.py --run_date 2026-10-07
```

Flag	Meaning
<code>--master / --deploy-mode</code>	Cluster manager and where the driver runs
<code>--driver-memory, --executor-memory, --executor-cores, --num-executors</code>	Resource sizing
<code>--conf</code>	Any Spark config; <code>--properties-file</code> loads many from a file
<code>--py-files</code>	Extra Python modules/zip shipped to executors
<code>--files</code>	Files (config, lookup) copied to each container's working directory
<code>--packages / --jars</code>	Maven dependencies / jar files

Reading a config file inside the script

```
import json
from pyspark import SparkFiles

# cluster mode: --files puts the file in the container working directory
with open("config.json") as f:
    cfg = json.load(f)
# or, works wherever the file was shipped:
cfg = json.load(open(SparkFiles.get("config.json")))

source_path, target_table = cfg["source_path"], cfg["target_table"]
```

Other formats: `configparser` for `.ini/.conf`, `yaml.safe_load` for YAML. Never hard-code credentials in the config; use a secrets manager.

55. SCD Type 1 and Type 2

Slowly Changing Dimensions describe how a dimension table handles changed attributes.

	SCD Type 1	SCD Type 2
Behavior	Overwrite the old value	Add a new row and expire the old one
History	Not kept	Fully kept
Extra columns	None	<code>start_date</code> , <code>end_date</code> , <code>is_current</code>
Use when	Corrections, history not needed	Need to report as it was at that time

SCD Type 1 (SQL and PySpark)

```
MERGE INTO dim_emp t USING stg s ON t.emp_id = s.emp_id
WHEN MATCHED THEN UPDATE SET t.name = s.name, t.sal = s.sal
WHEN NOT MATCHED THEN INSERT (emp_id, name, sal) VALUES (s.emp_id, s.name, s.sal);

# Plain DataFrames (no Delta): keep target rows that are not in source, then add all source rows
unchanged = tgt.join(stg, "emp_id", "left_anti")
new_tgt = unchanged.unionByName(stg) # write to a NEW path, then swap
```

SCD Type 2 (SQL MERGE: expire old version and insert new version)

```
MERGE INTO dim_emp t
USING (
    SELECT s.emp_id AS merge_key, s.* FROM stg s -- rows to match (update/insert)
    UNION ALL
    SELECT CAST(NULL AS INT) AS merge_key, s.* -- new version of changed rows
    FROM stg s JOIN dim_emp d ON s.emp_id = d.emp_id AND d.is_current = true
    WHERE s.sal <> d.sal OR s.name <> d.name
) u
ON t.emp_id = u.merge_key AND t.is_current = true
WHEN MATCHED AND (t.sal <> u.sal OR t.name <> u.name) THEN
    UPDATE SET t.is_current = false, t.end_date = current_date() -- expire old row
WHEN NOT MATCHED THEN
    INSERT (emp_id, name, sal, start_date, end_date, is_current)
    VALUES (u.emp_id, u.name, u.sal, current_date(), NULL, true); -- new/changed version
```

SCD Type 2 (PySpark, any storage)

```
today = F.current_date()
d = dim.filter("is_current = true").alias("d")
s = stg.alias("s")
j = d.join(s, F.col("d.emp_id") == F.col("s.emp_id"), "full_outer")

changed = j.filter("d.emp_id IS NOT NULL AND s.emp_id IS NOT NULL "
                  "AND (d.sal <> s.sal OR d.name <> s.name)")
added    = j.filter("d.emp_id IS NULL")

expired = (changed.select("d.*").withColumn("end_date", today)
          .withColumn("is_current", F.lit(False)))
fresh    = (changed.select("s.*").unionByName(added.select("s.*"))
          .withColumn("start_date", today).withColumn("end_date", F.lit(None).cast("date"))
          .withColumn("is_current", F.lit(True)))
unchanged = d.join(changed.select(F.col("d.emp_id").alias("emp_id")), "emp_id", "left_anti")
history    = dim.filter("is_current = false")

new_dim = (history.unionByName(unchanged).unionByName(expired).unionByName(fresh))
# write to a NEW path, then swap (you cannot overwrite what you are reading)
new_dim.write.mode("overwrite").parquet("/dim/emp_v2")
```

56. CDC (Change Data Capture) implementation

CDC identifies rows that were inserted, updated or deleted in a source since the last run, so you process only the changes.

CDC method	How it works
Timestamp/version column	Pick rows with updated_at > last_run . Simple but cannot see deletes
Snapshot compare	Compare today's snapshot with the target by key and row hash. Detects inserts, updates and deletes
Log-based	Read the database transaction log (Debezium, GoldenGate, AWS DMS) and stream changes
Table-native	Delta Change Data Feed (readChangeFeed), Iceberg changelog views

```
SELECT COALESCE(s.id, t.id) AS id,
CASE
  WHEN t.id IS NULL THEN 'INSERT'
  WHEN s.id IS NULL THEN 'DELETE'
  WHEN md5(concat_ws('|', s.name, s.salary, s.dept))
    <> md5(concat_ws('|', t.name, t.salary, t.dept)) THEN 'UPDATE'
  ELSE 'NO_CHANGE'
END AS change_type
FROM source s FULL OUTER JOIN target t ON s.id = t.id;

def add_hash(df, cols):
    parts = [F.col(c).cast("string") for c in cols]
    return df.withColumn("h", F.sha2(F.concat_ws("||", *parts), 256))

cols = ["name", "salary", "dept"]
s, t = add_hash(source, cols).alias("s"), add_hash(target, cols).alias("t")

cdc = (s.join(t, F.col("s.id") == F.col("t.id"), "full_outer")
      .select(F.coalesce(F.col("s.id"), F.col("t.id")).alias("id"),
              F.when(F.col("t.id").isNull(), "INSERT")
              .when(F.col("s.id").isNull(), "DELETE")
              .when(F.col("s.h") != F.col("t.h"), "UPDATE")
              .otherwise("NO_CHANGE").alias("change_type")))
```

```

MERGE INTO target t USING source s ON t.id = s.id
WHEN MATCHED AND (t.name <> s.name OR t.salary <> s.salary) THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *
WHEN NOT MATCHED BY SOURCE THEN DELETE;

```

The hash comparison avoids writing one comparison per column. Combine this with SCD Type 2 when you need history.

57. Cumulative sum as of the last date of every month (and months with no transactions)

Aggregate to month level first, then take a running sum. The cumulative value on each row is the balance as of that month's last day.

```

WITH monthly AS (
  SELECT date_trunc('month', txn_date) AS m, SUM(amount) AS amt
  FROM transactions GROUP BY 1
)
SELECT last_day(m) AS month_end, amt AS month_total,
       SUM(amt) OVER (ORDER BY m) AS cumulative_sum
FROM monthly ORDER BY m;

```

What if a month has no transactions?

The query above **silently skips** that month. Generate a **calendar of months** and left join, filling gaps with 0. The cumulative sum then carries forward.

```

WITH monthly AS (
  SELECT date_trunc('month', txn_date) AS m, SUM(amount) AS amt FROM transactions GROUP BY 1
),
bounds AS (SELECT MIN(m) AS mn, MAX(m) AS mx FROM monthly),
cal AS (SELECT explode(sequence(mn, mx, interval 1 month)) AS m FROM bounds)
SELECT last_day(c.m) AS month_end,
       COALESCE(mo.amt, 0) AS month_total,
       SUM(COALESCE(mo.amt, 0)) OVER (ORDER BY c.m) AS cumulative_sum
FROM cal c LEFT JOIN monthly mo ON c.m = mo.m
ORDER BY c.m;

monthly = (txn.groupBy(F.date_trunc("month", "txn_date").alias("m"))
           .agg(F.sum("amount").alias("amt")))
b = monthly.agg(F.min("m").alias("mn"), F.max("m").alias("mx"))
cal = b.select(F.explode(F.sequence("mn", "mx", F.expr("interval 1 month"))).alias("m"))
res = (cal.join(monthly, "m", "left").fillna(0, ["amt"]))
       .withColumn("cum", F.sum("amt").over(Window.orderBy("m")))
       .withColumn("month_end", F.last_day("m"))

```

58. Monthly sales report including months with no sales

Build a list of all 12 months first, then left join sales. **Put the year filter in the ON clause**, not in **WHERE**; a **WHERE** filter would turn the left join into an inner join and drop the empty months again.

```

WITH months AS (SELECT explode(sequence(1, 12)) AS month_num)
SELECT m.month_num,
       date_format(make_date(2026, m.month_num, 1), 'MMMM') AS month_name,
       COALESCE(SUM(s.amount), 0) AS total_sales
FROM months m
LEFT JOIN sales s ON month(s.sale_date) = m.month_num AND year(s.sale_date) = 2026
GROUP BY m.month_num
ORDER BY m.month_num;

(df.filter(F.year("sale_date") == 2026)
 .withColumn("month", F.month("sale_date"))
 .groupBy(F.lit(2026).alias("year"))
 .pivot("month", list(range(1, 13)))           # listing values forces all 12 columns to exist
 .sum("amount").fillna(0))

```

59. Delete records older than 30 days

```
DELETE FROM transactions WHERE txn_date < date_sub(current_date(), 30);
-- PostgreSQL/MySQL: WHERE txn_date < CURRENT_DATE - INTERVAL '30 days'
```

- Row-level **DELETE** works on **Delta, Iceberg, Hudi, Hive ACID** tables, not on plain Parquet.
- If the table is **partitioned by date**, drop whole partitions: **ALTER TABLE t DROP PARTITION (dt < '2026-09-07')** (fastest, no rewrite).
- Plain files: filter and write to a **new path**, then swap.

```
(df.filter(F.col("txn_date") >= F.date_sub(F.current_date(), 30))
.write.mode("overwrite").parquet("/data/txn_clean"))
```

60. Three consecutive absences

Classic **gaps-and-islands**. For consecutive dates, **date - row_number** is constant, so every streak forms one group.

```
WITH absent AS (
  SELECT student_id, att_date,
         ROW_NUMBER() OVER (PARTITION BY student_id ORDER BY att_date) AS rn
  FROM attendance WHERE status = 'Absent'
)
SELECT student_id, MIN(att_date) AS start_date, MAX(att_date) AS end_date, COUNT(*) AS days_absent
FROM absent
GROUP BY student_id, date_sub(att_date, rn)
HAVING COUNT(*) >= 3;
```

Alternative with LAG (when every day has a row)

```
SELECT DISTINCT student_id FROM (
  SELECT student_id, status,
         LAG(status, 1) OVER (PARTITION BY student_id ORDER BY att_date) AS p1,
         LAG(status, 2) OVER (PARTITION BY student_id ORDER BY att_date) AS p2
  FROM attendance
) t WHERE status = 'Absent' AND p1 = 'Absent' AND p2 = 'Absent';
```

If weekends/holidays have no rows, consecutive working days need a calendar table to define "next day".

61. Handling double delimiters in input files

Example record: **101||John||Sales**.

```
# Spark 3+ accepts a multi-character separator
df = spark.read.option("sep", "||").option("header", True).csv("file.txt")

# Fallback: read as text and split on an escaped regex
raw = spark.read.text("file.txt")
parts = F.split("value", r"\\|\\", -1) # -1 keeps trailing empty fields
df = raw.select(parts[0].alias("id"), parts[1].alias("name"), parts[2].alias("dept"))
```

Say in the interview: **|** is a regex special character, so escape it in **split**. Use limit **-1** so empty trailing values are not dropped, and check for delimiters inside quoted data.

62. Identify and remove malformed records (permissive modes)

Mode	Behavior
PERMISSIVE (default)	Keeps the row, sets bad fields to NULL and stores the raw line in a corrupt-record column
DROPMALFORMED	Silently drops bad rows
FAILFAST	Throws an exception on the first bad row

Best practice: use **PERMISSIVE**, capture the bad rows in a **quarantine** location, and continue with the good ones.

```
from pyspark.sql.types import *
schema = StructType([StructField("id", IntegerType()), StructField("name", StringType()),
                      StructField("salary", DoubleType()),
                      StructField("_corrupt_record", StringType())])

df = (spark.read.schema(schema).option("mode", "PERMISSIVE")
      .option("columnNameOfCorruptRecord", "_corrupt_record").csv("input.csv"))

df.cache() # needed: Spark blocks queries that touch only the corrupt column of a raw file
bad = df.filter("_corrupt_record IS NOT NULL")
good = df.filter("_corrupt_record IS NULL").drop("_corrupt_record")
bad.write.mode("overwrite").csv("/quarantine/bad_records")
```

On Databricks you can also use **badRecordsPath**. Add validation rules (null checks, range checks, column-count checks).

63. Process a specific file from a directory

```
df = spark.read.option("header", True).csv("/data/input/sales_2026-10-07.csv") # exact file
df = spark.read.option("header", True).csv("/data/input/sales_2026-10-*.csv") # glob pattern
df = spark.read.csv(sys.argv[1], header=True) # name passed as an argument
df = df.withColumn("src_file", F.input_file_name()) # which file each row came from
```

To pick the latest file, list the directory (**hdfs dfs -ls**, **dbutils.fs.ls**, **boto3**) and choose by name or modification time.

PART 5: Linux and Shell Scripting

64. Output/effect of common commands: cat >, chmod 755, ls -ltr >

Command	What it does
<code>cat file1.txt file2.txt > next.txt</code>	Concatenates file1 then file2 and writes to <code>next.txt</code> (creates or overwrites it). <code>>></code> would append.
<code>chmod 755 file1.txt</code>	Owner: read+write+execute (7); group: read+execute (5); others: read+execute (5)
<code>ls -ltr directory_path > new.txt</code>	Long listing sorted by modified time, oldest first , redirected to <code>new.txt</code> (nothing printed on screen)

What does `ls -ltr` mean?

- `-l` long format (permissions, owner, size, date), `-t` sort by modification time (newest first), `-r` reverse the order. Together `-ltr` **shows the most recently modified file last**, which is handy for finding the latest file or log.

65. chmod: permission scenarios

Permissions are r=4, w=2, x=1, added per class: **owner, group, others**.

Mode	Meaning	Typical use
<code>755</code>	<code>rxr-xr-x</code>	Scripts and directories everyone can run/read
<code>644</code>	<code>rw-r--r--</code>	Normal files and configs
<code>600</code>	<code>rw-----</code>	Private files (keys, credentials)
<code>700</code>	<code>rwX-----</code>	Private scripts/directories
<code>777</code>	<code>rxwxrwxrwx</code>	Everyone full access (avoid; security risk)

```
chmod +x run.sh          # add execute for everyone
chmod u+x,g-w,o-r file    # symbolic: owner +x, group -w, others -r
chmod -R 755 /app/scripts # recursive
ls -l run.sh              # verify: -rxr-xr-x
```

66. Rename multiple files; shell script to rename with backup_ prefix

```
#!/bin/bash
# rename every file in a directory to backup_<actual_filename>
dir="/data/input"
for f in "$dir"/*; do
    [ -f "$f" ] || continue          # skip sub-directories
    base=$(basename "$f")
    mv "$f" "$dir/backup_$base"
done

for f in *.txt; do mv "$f" "${f%.txt}.csv"; done    # change extension .txt -> .csv
for f in *.log; do mv "$f" "$(date +%F)_$f"; done  # add a date prefix
rename 's/old/new/' *.txt                  # Perl rename (if installed)
```

Always quote `"$f"` so names with spaces work, and test with `echo mv ...` first.

67. Read the first line of each of 10 files into a new file

```
#!/bin/bash
dir="/data/input"
out="/data/first_lines.txt"
> "$out" # create/empty the output file
for f in "$dir"/*; do
    [ -f "$f" ] && head -n 1 "$f" >> "$out"
done

for f in /data/input/*; do head -n 1 "$f"; done > first_lines.txt # one-liner
awk 'FNR==1 {print}' /data/input/* > first_lines.txt # awk version
```

68. grep command, searching a word in a file

```
grep "word" filename.csv # lines containing word
grep -i "word" file # case-insensitive
grep -n "word" file # show line numbers
grep -c "word" file # count matching lines
grep -v "word" file # lines NOT containing word
grep -w "word" file # whole word only
grep -E "foo|bar" file # extended regex (OR)
grep -r "word" /path/dir # recursive through a directory
grep -A2 -B2 "word" file # 2 lines after and before the match
```

Note: `-r` is for directories. For one file, `grep "word" filename.csv` is enough.

69. List all files containing a specific word; print files and directories with a word

Files whose CONTENT contains the word

```
grep -rl "word" /path/to/dir # -r recursive, -l print only file names
grep -rli "word" /path/to/dir # case-insensitive
grep -rn "word" /path/to/dir # file:line:text (shows the matching lines)
```

Files and directories whose NAME contains the word (including subfolders)

```
find /path -iname "*word*" # files AND directories
find /path -type d -iname "*word*" # directories only
find /path -type f -iname "*word*" # files only
hdfs dfs -ls -R /path | grep word # same idea on HDFS

import os
for root, dirs, files in os.walk("/path"):
    for d in dirs:
        if "word" in d.lower(): print("DIR :", os.path.join(root, d))
    for f in files:
        if "word" in f.lower(): print("FILE:", os.path.join(root, f))
```

70. Commonly used Unix commands

Task	Commands
Navigate/list	<code>pwd</code> , <code>cd</code> , <code>ls -lh</code> , <code>tree</code>
View files	<code>cat</code> , <code>head -n 5</code> , <code>tail -f log</code> (follow live), <code>less</code> , <code>wc -l</code>
Copy/move/delete	<code>cp -r</code> , <code>mv</code> , <code>rm -rf</code> (careful), <code>mkdir -p</code>
Search/filter	<code>grep</code> , <code>find</code> , <code>awk</code> , <code>sed</code> , <code>cut -d',' -f1</code> , <code>sort</code> , <code>uniq -c</code>
Disk/process	<code>df -h</code> , <code>du -sh *</code> , <code>ps -ef grep x</code> , <code>top</code> , <code>kill -9 <pid></code>
Permissions	<code>chmod</code> , <code>chown</code>
Transfer/schedule	<code>scp</code> , <code>rsync</code> , <code>curl</code> , <code>crontab -e</code>
HDFS	<code>hdfs dfs -ls/-put/-get/-cat/-rm/-du -h</code>

```
cut -d',' -f2 data.csv | sort | uniq -c | sort -nr | head # top values of column 2
```

71. How to check the status of a Spark job and its logs

Need	Command / place
List running apps (YARN)	<code>yarn application -list -appStates RUNNING</code>
Status of one app	<code>yarn application -status <application_id></code>
Kill an app	<code>yarn application -kill <application_id></code>
Logs of a finished app	<code>yarn logs -applicationId <application_id></code>
Only driver/AM log	<code>yarn logs -applicationId <id> -am 1</code>
Live progress	Spark UI (link in the YARN Resource Manager) or the History Server after completion
Client mode	Driver log prints on the terminal; redirect with <code>nohup spark-submit ... > job.log 2>&1 &</code>
Kubernetes	<code>kubectl logs <driver-pod></code>
Orchestrator	Airflow task log in the UI

```
yarn application -list -appStates RUNNING | grep my_etl_job
yarn logs -applicationId application_1696600000000_0042 | grep -i "error\|exception"
```

PART 6: Python and Project Questions

72. Static method vs Class method in Python

	@staticmethod	@classmethod
First parameter	None (no self, no cls)	<code>cls</code> (the class itself)
Access to class state	No	Yes (class variables, other class methods)
Access to instance state	No	No
Typical use	Utility function that logically belongs to the class	Alternative constructors, factory methods, change class-level data
Works with subclasses	Behaves the same	<code>cls</code> becomes the subclass, so it is polymorphic

```
class Employee:
    company = "Acme"

    def __init__(self, name, sal):
        self.name, self.sal = name, sal

    @classmethod
    def from_string(cls, s):
        name, sal = s.split("-")
        return cls(name, int(sal))

    @staticmethod
    def is_valid_salary(sal):
        return sal > 0

e = Employee.from_string("aa-1000")
Employee.is_valid_salary(500)
```

alternative constructor

pure utility, no class/instance data needed

True

An **instance method** (the normal one) takes `self` and can read/modify that object's data.

73. CI/CD for a data engineering project

- **CI (Continuous Integration)**: on every pull request, automatically **lint** (flake8/black), run **unit tests** (pytest with a local SparkSession) and build the package.
- **CD (Continuous Delivery/Deployment)**: after merge, automatically **deploy** code (wheel/zip, notebooks, DAGs) to dev, then test/UAT, then prod, with approval gates.
- **Tools**: Git (GitHub/GitLab/Azure Repos) + Jenkins / GitHub Actions / Azure DevOps / GitLab CI; Terraform/ARM for infrastructure; Docker if used.

```
# Typical pipeline stages (example)
1. Developer pushes feature branch -> pull request
2. CI: install deps -> flake8/black -> pytest (unit + small Spark tests) -> build wheel
3. Merge to main -> deploy to DEV (copy jar/wheel + DAGs, update job config)
4. Integration tests on DEV data -> promote to QA/UAT -> manual approval -> PROD
5. Monitor, alert, roll back to previous version if the release fails
```

Say in the interview: Mention config per environment (dev/qa/prod), secrets in a vault, versioned artifacts, and automated rollback.

74. Agile methodology

Agile delivers software in **short iterations** with continuous feedback instead of one big release. The most common framework is **Scrum**.

Element	Details
Roles	Product Owner (priorities), Scrum Master (process), Development team
Sprint	Fixed 1-4 week iteration that ends with a working increment
Artifacts	Product backlog, sprint backlog, user stories with story points
Ceremonies	Sprint planning, daily stand-up (15 min), sprint review (demo), retrospective
Tools	Jira / Azure Boards for stories, tasks, boards

How it looks for a data engineer: a story such as "Load daily orders into the warehouse" is estimated, broken into tasks (ingest, transform, test, deploy), developed and reviewed within the sprint, then demoed with the data delivered.

75. Explaining your last project (answer framework)

Use a clear structure so the answer is short and confident: **Context, Architecture, Your role, Challenges, Result.**

Part	What to say
Context	Business problem and data volume. Example: "Daily sales reporting for 50M rows/day from 5 source systems"
Architecture	Source > ingestion (Kafka/ADF/Sqoop) > raw zone (Parquet) > PySpark transformations > curated tables (Hive/Delta) > BI/Reporting. Orchestration with Airflow
Your role	The pieces you built: ingestion jobs, transformation logic, SCD2 dimensions, CDC, tuning, CI/CD, monitoring
Challenges	Data skew, small files, late-arriving data, schema changes, OOM errors, and how you fixed each
Result	Numbers: runtime reduced from 3 h to 45 min, cost saved, SLA met, data quality improved

Say in the interview: Prepare one or two concrete stories with metrics (before/after) for a performance problem, because follow-up questions will dig into them.

76. Project folder structure

```
project/
|-- src/
|   |-- jobs/           # one file per pipeline (ingest_orders.py, build_dim_customer.py)
|   |-- transformations/ # reusable DataFrame functions
|   |-- utils/          # logging, spark_session.py, config loader, schema definitions
|   |-- main.py         # entry point (parses args, calls the job)
|-- config/
|   |-- dev.json
|   |-- qa.json
|   |-- prod.json
|-- tests/              # pytest unit tests (small local Spark DataFrames)
|-- sql/                # DDL and reusable SQL
|-- dags/               # Airflow DAGs
|-- scripts/            # shell scripts (spark-submit wrappers, deployment)
|-- notebooks/          # exploration only
|-- requirements.txt / setup.py / pyproject.toml
|-- Jenkinsfile or .github/workflows/ # CI/CD definition
-- README.md
```

Explain the **why**: config separated from code per environment, transformations separated from I/O so they are unit-testable, one job per file for clear ownership, and CI/CD at the root.